

Estensione del laboratorio IoT - Gestione di 10 sensori e dashboard multi-sensore

Tecnologie: Docker, MQTT, Mosquitto, Node-RED, MailHog

Livello: Junior / Entry-level

Approccio: DevOps, step incrementali

Obiettivo dell'estensione

Estendere lo stack IoT realizzato nei precedenti step per:

- simulare **10 sensori IoT di temperatura**
- ogni sensore deve:
 - essere **un container separato**
 - inviare la temperatura ogni **X secondi**
 - usare MQTT come protocollo di comunicazione
- visualizzare i dati di **tutti i sensori** su una **dashboard unica**
- mantenere lo stack **scalabile e manutenibile**

N.B. L'architettura deve rimanere **event-driven** e non devono essere introdotti servizi esterni o cloud.

Requisiti obbligatori

◆ Sensori IoT

Ogni sensore deve:

- essere avviato come **container Docker**
- usare **lo stesso Dockerfile**
- differenziarsi tramite **variabili di ambiente**
- pubblicare su topic MQTT **gerarchici**

Esempio di topic richiesto:

- **iot/aula/sensor01/temperatura**
- **iot/aula/sensor02/temperatura**
- ...
- **iot/aula/sensor10/temperatura**

N.B: Non creare 10 copie dello script Python, Il codice deve essere **riutilizzabile**

◆ Docker Compose

Il file `docker-compose.yml` deve:

- contenere:
 - Mosquitto
 - Node-RED
 - MailHog
 - **10 servizi sensore**
- usare **un solo build** per i sensori
- usare **variabili di ambiente** per:
 - `SENSOR_ID`
 - `SEND_INTERVAL`
 - `MQTT_BROKER`
 - `MQTT_PORT`

N.B. i servizi possono differire **solo per environment e nome**

◆ Dashboard

La dashboard deve:

- essere realizzata con **FlowFuse Dashboard**
- mostrare **la temperatura di tutti i sensori**
- usare **componenti separati** per ogni sensore:
 - gauge multipli
 - oppure lista / tabella aggiornata
- essere leggibile e ordinata

N.B. Deve essere possibile capire **quale sensore** sta inviando il dato.

Requisiti di qualità

- Uso corretto di:
 - wildcard MQTT (+)
 - topic gerarchici
- Stack **scalabile**
- Nessun valore hard-coded nel codice Python
- Flussi Node-RED chiari e ordinati
- Commit incrementali e descrittivi

Cosa NON è permesso

- ✗ Copiare lo script Python 10 volte
- ✗ Usare IP al posto dei nomi servizio Docker
- ✗ Usare la dashboard deprecata
- ✗ Usare servizi esterni (SMTP reali, cloud, ecc.)

Verifiche richieste allo studente

Lo studente deve dimostrare che:

- tutti i 10 sensori sono attivi (`docker ps`)
- i dati arrivano correttamente al broker
- Node-RED riceve i dati di tutti i sensori
- la dashboard mostra 10 valori distinti
- lo stack riparte correttamente dopo `docker compose down/up`

Consegna

La consegna consiste in:

- repository GitHub Classroom aggiornato
- `docker-compose.yml` funzionante
- codice dei sensori parametrico
- flow Node-RED configurato correttamente

N.B. **Non sono richiesti screenshot**, Il repository è la consegna finale

Messaggio di commit finale (CONSEGNA LAVORO)

```
git add .  
git commit -m "Consegna finale: stack IoT con 10 sensori, MQTT, Node-RED e MailHog"  
git push
```

Istruzioni per la consegna

Una volta completato **tutto lo stack** (10 sensori, dashboard e alert email):

1. Verifica che lo stack sia avviabile:
2. Verifica che:
 - a. i 10 sensori siano attivi
 - b. la dashboard mostri tutti i sensori
 - c. gli alert email funzionino

Checklist finale di autovalutazione

Prima di effettuare il *commit finale di consegna*, verifica di poter rispondere **Sì** a tutte le voci seguenti.

Se una risposta è **NO**, torna sul punto indicato e correggi lo stack.

♦ 1. Repository e versionamento

- ☐ Ho clonato il **mio repository personale** da GitHub Classroom
- ☐ Il repository contiene **solo il mio lavoro**
- ☐ Ho effettuato **commit incrementali**, uno per ogni step
- ☐ I messaggi di commit sono **chiari e descrittivi**
- ☐ Il commit finale ha un messaggio professionale di consegna

♦ 2. Struttura del progetto

- ☐ La struttura delle cartelle è ordinata e coerente
- ☐ È presente un unico `docker-compose.yml`
- ☐ I servizi sono definiti in modo leggibile
- ☐ Non ci sono file inutili o duplicati

♦ 3. Broker MQTT (Mosquitto)

- ☐ Mosquitto è avviato come **container Docker**
- ☐ Il container ascolta correttamente sulla porta MQTT
- ☐ Gli altri servizi si collegano usando il **nome del servizio Docker**
- ☐ Non ho installato Mosquitto sul sistema operativo

♦ 4. Sensori IoT (Publisher)

- ☐ Sono presenti **10 sensori distinti**
- ☐ Ogni sensore è un **container separato**
- ☐ Tutti i sensori usano **lo stesso codice Python**
- ☐ I sensori si distinguono tramite **variabili di ambiente**
- ☐ Ogni sensore pubblica sul proprio topic MQTT
- ☐ Il codice non contiene IP o valori hard-coded

Domanda di controllo

Riesco ad aggiungere un undicesimo sensore senza modificare il codice?

♦ 5. Topic MQTT e protocollo

- ☐ I topic sono **gerarchici e coerenti**
- ☐ I topic identificano chiaramente il sensore
- ☐ In Node-RED uso **wildcard MQTT** (+ o #)
- ☐ Il modello publish / subscribe è rispettato
- ☐ I sensori non conoscono i consumer

♦ 6. Node-RED come consumer

- ☐ Node-RED è avviato come container Docker
- ☐ Node-RED riceve correttamente i messaggi MQTT
- ☐ I flussi sono ordinati e comprensibili
- ☐ La logica applicativa **non è nei sensori**

Domanda di controllo

Se cambio la logica di alert, devo modificare i sensori?

♦ 7. Dashboard (FlowFuse Dashboard)

- ☐ Uso la dashboard **FlowFuse**
- ☐ La dashboard è accessibile via browser
- ☐ Visualizzo **tutti i 10 sensori**
- ☐ Ogni sensore è chiaramente identificabile
- ☐ La dashboard è leggibile e ordinata

♦ 8. Alert email e MailHog

- ☐ Ho aggiunto MailHog allo stack Docker
- ☐ Node-RED invia email solo quando la soglia è superata
- ☐ L'oggetto dell'email include la temperatura rilevata
- ☐ Il corpo dell'email è chiaro e informativo
- ☐ Nessuna email reale viene inviata all'esterno

Domanda di controllo

Se cambio la soglia, devo modificare il sensore?

♦ 9. Architettura e concetti IoT

- ☐ So spiegare cos'è un **broker MQTT**
- ☐ So spiegare la differenza tra **publisher e subscriber**
- ☐ So spiegare perché questa architettura è **event-driven**
- ☐ So spiegare perché MQTT è più adatto di HTTP
- ☐ So spiegare come lo stack può scalare

♦ 10. Verifica finale dello stack

- ☐ Lo stack parte correttamente con **docker compose up**, oppure con **docker compose build**?
- ☐ Tutti i container risultano attivi (**docker ps**)
- ☐ I dati fluiscono dai sensori alla dashboard
- ☐ Gli alert email arrivano correttamente
- ☐ Nessun errore bloccante nei log

✓ Autovalutazione finale

- ☐ Ho compreso **il ruolo dei protocolli IoT**
- ☐ Ho compreso **l'importanza del broker**
- ☐ Ho compreso **il concetto di scalabilità**
- ☐ Mi sento in grado di **spiegare il progetto a voce**
- ☐ Il progetto rappresenta un lavoro **professionale**

Docker Build quando serve

Se il progetto contiene servizi con `build:` (come i sensori Python), è necessario usare `--build` almeno una volta.

Nel nostro stack:

- ☒ Mosquitto → `image:` (non serve build)
- ☒ Node-RED → `image:` (non serve build)
- ☒ MailHog → `image:` (non serve build)
- ☒ Sensori → `build: ./sensor` ☒ **SERVE build**

Comportamento reale di Docker Compose

- Prima esecuzione (obbligatoria)

Alla **prima esecuzione** o **dopo modifiche al codice Python / Dockerfile**, il comando corretto è: **`docker compose up --build`**

Questo:

- costruisce l'immagine del sensore
- include il codice Python aggiornato
- poi avvia i container

N.B. **Senza `--build` il container usa eventualmente un'immagine vecchia o inesistente**

- Esecuzioni successive (senza modifiche)

Se **non hai cambiato nulla** in:

- `sensor.py`
- `Dockerfile`
- `docker-compose.yml`

allora è corretto e sufficiente : **`docker compose up`**

Docker:

- riusa le immagini già buildate
- è più veloce
- è il comportamento tipico in produzione

Domande per la vs. riflessione

1. Perché ogni sensore è un container separato?

Ogni sensore è un container separato perché rappresenta un **dispositivo indipendente**.

In questo modo possiamo:

- simulare più dispositivi reali
- scalare il sistema aggiungendo o rimuovendo sensori
- isolare i malfunzionamenti di un singolo sensore
- distribuire i carichi in modo indipendente

Questo segue il modello **microservizi**.

- Cosa succede se un sensore smette di funzionare? Gli altri devono continuare a inviare dati.
- Un container = un processo = una responsabilità.
- Se tutti i sensori fossero nello stesso container, non potresti scalarli o riavviarli singolarmente.

2. Come scala questo stack a 100 sensori?

Lo stack scala facilmente perché:

- MQTT usa un modello **publish/subscribe**
- i sensori non conoscono i consumer
- il broker gestisce la distribuzione dei messaggi
- i sensori sono containerizzati e parametrizzati

Per passare da 10 a 100 sensori basta:

- aggiungere nuovi servizi
- senza modificare broker o Node-RED

Suggerimenti:

- Non devi cambiare il codice di Mosquitto o Node-RED per aggiungere un sensore.

- I topic MQTT permettono di gestire molti sensori senza modificare il consumer.
- Lo scaling avviene aggiungendo publisher, non cambiando il protocollo.

3. Perché Node-RED non è un broker?

Node-RED non è un broker perché:

- non gestisce connessioni MQTT multiple
- non distribuisce messaggi tra publisher e subscriber
- non garantisce disaccoppiamento

Il suo ruolo è:

- **consumare eventi**
- applicare logica
- integrare servizi (dashboard, email)

Il broker è Mosquitto.

Suggerimenti:

- Node-RED riceve messaggi, ma non li smista.
- Senza Mosquitto i sensori non saprebbero dove inviare i dati.

4. Qual è il vantaggio di MQTT rispetto a HTTP?

MQTT è più adatto di HTTP perché:

- è **event-driven**
- è **asincrono**
- riduce il consumo di rete
- permette **disaccoppiamento totale**

Con HTTP ogni sensore dovrebbe:

- sapere a chi inviare i dati
- gestire errori di rete Con MQTT questo problema non esiste.

5. Dove aggiungeresti la sicurezza?

La sicurezza può essere aggiunta:

- sul broker MQTT (autenticazione e autorizzazione)
- con TLS (porta 8883)
- con utenti e ACL sui topic

I sensori non devono gestire logica di sicurezza complessa, ma solo configurazione.

6. Perché la logica di alert non è nel sensore?

La logica di alert non è nel sensore perché:

- il sensore deve fare **una sola cosa**: inviare dati
- la logica può cambiare senza aggiornare i dispositivi
- il consumer (Node-RED) è il posto giusto per applicare regole

Questo rende il sistema:

- più flessibile
- più manutenibile

7. Perché questo sistema è definibile “event-driven”?

È event-driven perché:

- i sensori generano **eventi**
- gli eventi vengono pubblicati
- i consumer reagiscono agli eventi
- non c'è comunicazione diretta tra componenti

Ogni azione avviene **in risposta a un evento**, non a una richiesta.

